

---

# Which Regression Model Predicts Student Exam Scores Best — and Which One Survives Messy Data?

---

[Your Name] — [School, City, Country] — [Month Year]

## Introduction

---

As an IGCSE student applying to university, I faced many challenges. One of them is that I have to put predicted grades in my admission documents, and these predictions may turn out to be wrong at the end of the day — and I could lose the university offer because of it. That is why I thought of AI regression models predicting my grades, and I wanted to make sure that they really are useful. That is why I made this experiment.

This experiment focuses mainly on three things. First, the model gets messy data that contains errors and noise, because I can't just give the model a clean dataset, train it, and expect the result to reflect real life. That isn't how real life works. In real life, datasets contain a percentage of errors: some students may skip some fields, others may write wrong data, or make human errors while writing. No one in the world can guarantee a clean dataset. So our model must be able to deal with bad datasets correctly, and when given a clean dataset the change in results must never be big — because any big change demonstrates that the model really suffers from messy data. Finally, our model must understand the relationships between variables correctly.

In this experiment we have a messy dataset of 5,050 rows, where we set some statistical rules for it and fill missing values with the median of the column. Then we pass this dataset to the three models our experiment focuses on: SVR (Support Vector Regression), multiple linear regression, and random forest regression. We measure model performance by the following. First we have MAE (mean absolute error), which calculates the average absolute error per student — for example, "6 points off." We also have  $R^2$ , which says how much of the variation in scores the model explains. After that, we give the models the real zero-errors dataset and see the change in MAE, and whether it is large. Then we give the models the raw messy file with no edits at all and see the MAE. After all these tests, we calculate each model's Recovery, which measures how much of the damage caused by the messy data our cleaning repaired.

## Data and Cleaning

---

The first step is understanding the messy dataset we received. The average sleep for students is 6 hours per night, which is a sensibly low number, and the mean study time is 12 hours per week — about 1.7 hours per day — which is very low. This pushed us to take a deeper look inside the data for anomalies, and we found several types of errors: duplicated students, impossible values, inconsistent categorical data, and missing data.

Now, enough explaining the problem — let's start cleaning. First we dropped all duplicates in the dataset, which removed about 50 students who were duplicated. Then we set some statistical rules for our dataset. Any student who studied more than 75 hours per week, or has more than 100% attendance, or less than 0 hours of sleep — we count these as data-entry errors. But here is the thing: we don't delete the whole student. We just turn the wrong value into an empty value and later fill it with the median of the column. So no student is deleted from our dataset except the 50 duplicates, and every student keeps the rest of their real data. We did not remove students with 0 study hours, because some students genuinely don't study, and their low marks prove this is real behavior, not an error.

For categorical data such as internet quality and school type, we converted the whole column to lowercase, because models aren't smart enough to understand that "Public" is the same thing as "public." From the model's perspective it only sees numbers, so after encoding, "Public" and "public" would become two totally different columns, and that damages the model's performance.

For missing data, we fill each empty value with the median of the whole column. We picked the median and not the mean because the dataset has extreme wrong values that would pull the mean away, while the median doesn't care about them. Through our data analysis, these are the fields with empty values: sleep hours, previous GPA, screen time per day, study hours per week, and attendance. Another thing we discovered: the missing sleep data isn't random at all — students who are missing sleep data have lower attendance than the rest. That means our median fill takes values mostly from the engaged students, which is a small bias we will mention in the limitations. Also, for internet quality we found 16 empty values, and the funny part is they weren't really empty — the label "None" gets automatically read as a missing value when loading the file, so these students really had "none" internet, and we just restored their real value instead of guessing.

Now that we have finished cleaning our messy dataset, it's time to visualize our data.

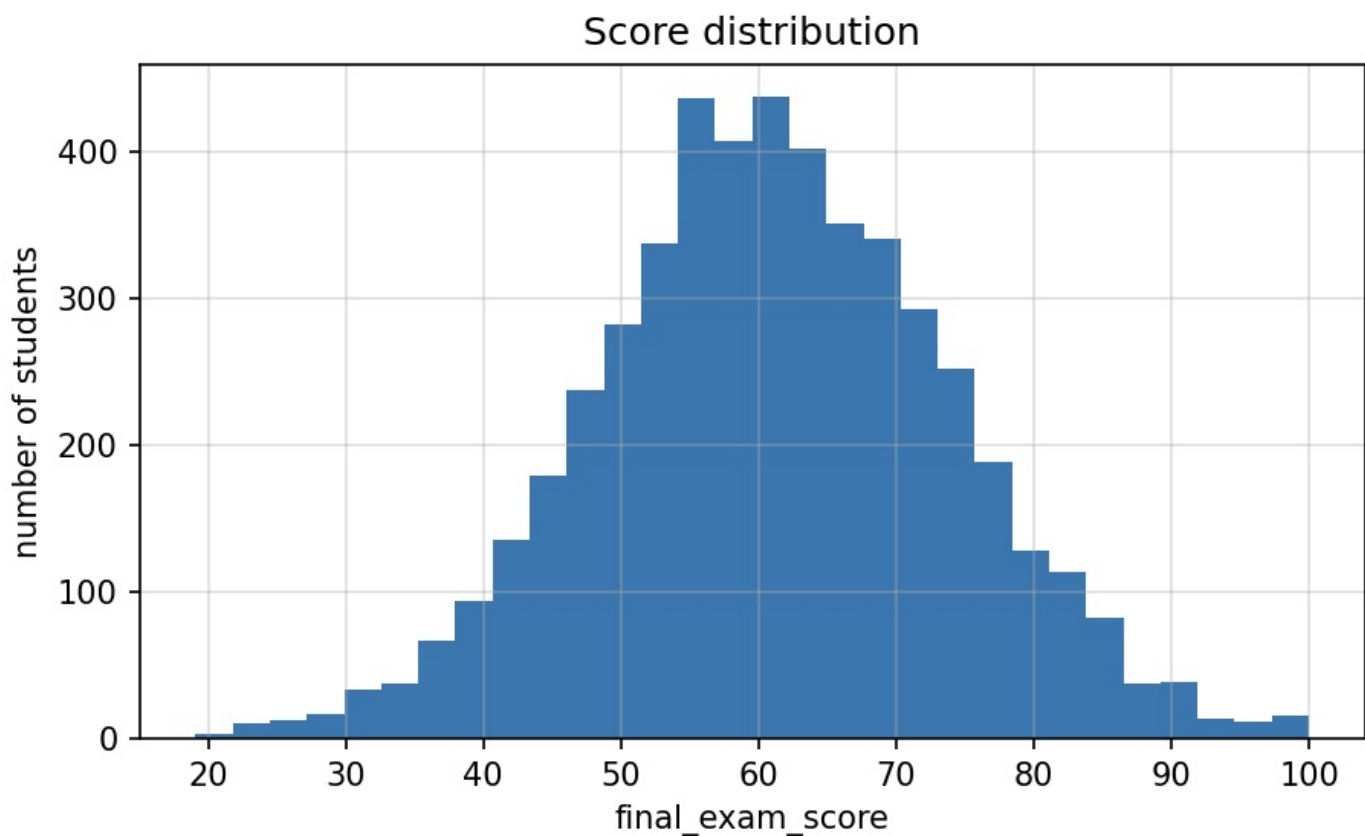


Figure 1: Score distribution histogram of the cleaned messy dataset.

Figure 1 is the score distribution histogram, which shows the frequency of the scores. As we can see, the highest frequency throughout the histogram is from 50 to 70 out of 100.

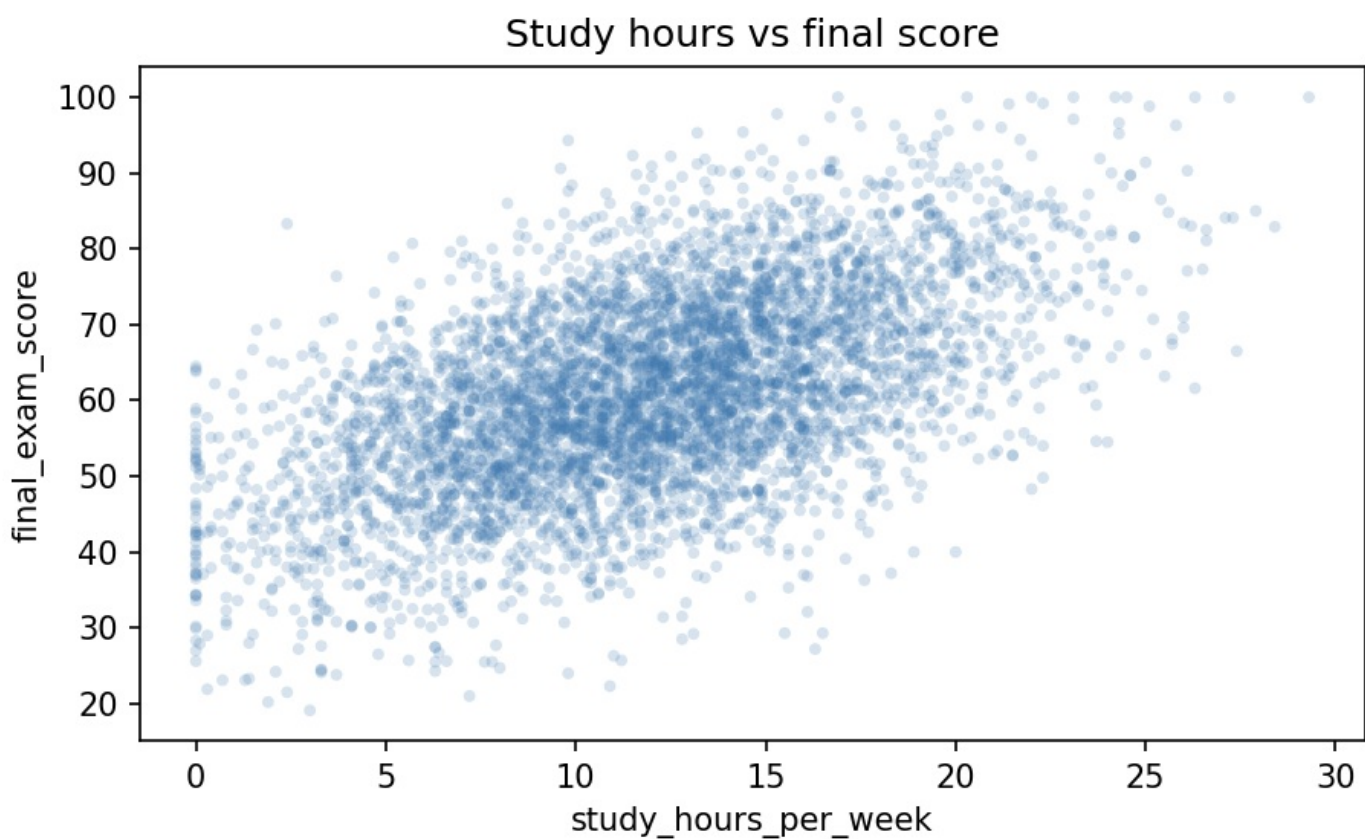


Figure 2: Study hours per week vs final exam score.

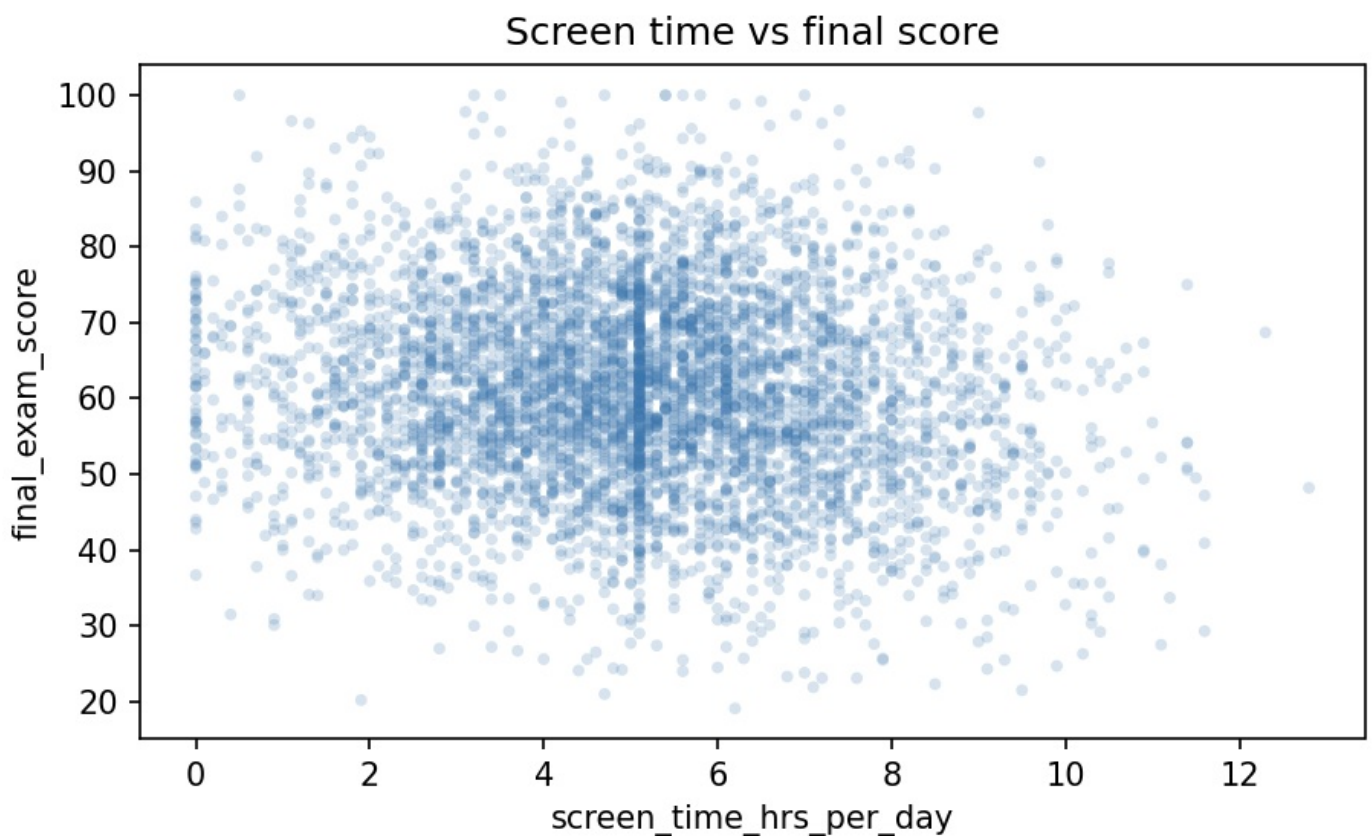


Figure 3: Screen time per day vs final exam score.

From Figures 2 and 3 we can conclude something important. There is a correlation between the final exam score and study hours per week, which is obviously sensible — we can draw an increasing linear line through those points. In the screen-time graph we can see a negative trend, which suggests the lower the screen time, the higher the mark. But these single-variable trends are informative and incomplete at the same time: as we will show later, the effect of study hours actually depends on sleep — a relationship that no single scatter plot can reveal on its own.

## Method

Now we split our dataset into the dependent variable (y) and independent variables (X). First we dropped some columns from X: student\_id, final exam score, and passed. This last one is important, because “passed” is calculated directly from the final exam score, so keeping it would leak the answer into the features and give fake good results. Then we encoded the categorical columns so the model is able to interpret them. Our only dependent variable is the final exam score. The train set is 80% of the data and the test set is 20%, with a random state of 42 so our results are reproducible on every run.

Now here is the thing: our experiment runs this exact same pipeline three times, not once. The first run is on the messy dataset after our cleaning. The second run is on the clean zero-errors dataset. And the final run is on the raw messy file with no cleaning at all — for this run we just fill any empty value with -999 so the code can run, exactly like what a careless analyst would do. Same models, same split, same metrics every single time; the only thing that changes is the data itself. After all three runs we calculate the Recovery for each model:

$$\text{Recovery\%} = (\text{MAE}_{\text{raw}} - \text{MAE}_{\text{messy}}) / (\text{MAE}_{\text{raw}} - \text{MAE}_{\text{clean}}) \times 100$$

which tells us how much of the damage caused by the messy data our cleaning repaired.

For every run we also make a dummy regressor model as our baseline to compare the MAE values of the other models. Because if a model gave an MAE of 19, for example — how would I say good or bad? There must be a baseline. Here comes the dummy regressor into the picture. The dummy regressor always predicts the average score, so it represents “knowing nothing.” Each condition gets its own baseline: the dummy gave an MAE of 10.26 on the cleaned messy data, 10.32 on the clean data, and 10.68 on the raw data, with  $R^2$  of 0 every time — which makes sense, because a model that always guesses the average explains nothing. Each model in our research gets an MAE score and an  $R^2$  score, and we compare them to the baseline of that condition and to the other models.

Our settings for the models are the following. For linear regression, nothing to change. For SVR, we scale the features first, because SVR is very sensitive to the sizes of numbers, and we give it a C (how strict the model is with errors) of 100 with an RBF kernel — and later we test 4 more C values to see how much this setting really matters. Random forest regression has 300 trees, and no scaling here, because the forest doesn’t care about scale — it only asks whether the value is above or below a threshold. Let’s start feeding them with data.

## Results

### Run 1 — cleaned messy data

After training each model with our cleaned data, these were the results. Multiple linear regression gave an MAE of 6.84 and an  $R^2$  of 0.565. These numbers are really good compared to the baseline of 10.26: a model that predicts about 7 marks off can give useful estimations, and it explains 56.5% of the variation in scores. Now to SVR — a note here: we did standard scaling on the data, because SVR can be influenced by number size, and of course we don't want that to happen — with  $C = 100$ . SVR gave an MAE of 8.18 and  $R^2$  of 0.376. This result actually surprised us; we always thought SVR would beat multiple linear regression. Compared to the baseline this number is still good, not bad, but multiple linear regression stays on top of the leaderboard. Because of this surprise, we tested 4 more  $C$  values:  $C = 1$  gave an MAE of 7.00,  $C = 10$  gave 7.02,  $C = 300$  gave 8.89, and  $C = 1000$  gave 9.91 — almost as bad as the dummy. So the  $C$  setting mattered more than we expected: the same algorithm ranged from 7.0 to 9.9 depending on one setting. Finally, random forest regression with 300 trees gave an MAE of 7.19 and  $R^2$  of 0.523. This model was the nearest to multiple linear regression, but at the end of the day it was still defeated in both MAE and  $R^2$ .

#### Scoreboard for the cleaned messy data:

| Model                      | Our expected rank | Actual MAE | $R^2$ |
|----------------------------|-------------------|------------|-------|
| Multiple linear regression | 3rd               | 6.84       | 0.565 |
| SVR ( $C=100$ )            | 1st               | 8.18       | 0.376 |
| Random forest regression   | 2nd               | 7.19       | 0.523 |
| Baseline (dummy)           | —                 | 10.26      | 0.00  |

This broke all our expectations as a team. We all thought the winner would be between SVR and random forest, but to our surprise it was multiple linear regression.

### Why did linear regression win? The sleep-study interaction

We took a deeper look into the dataset and plotted more graphs to understand why we got these results, and we found a strong factor: sleep hours together with study hours influence the final exam score.

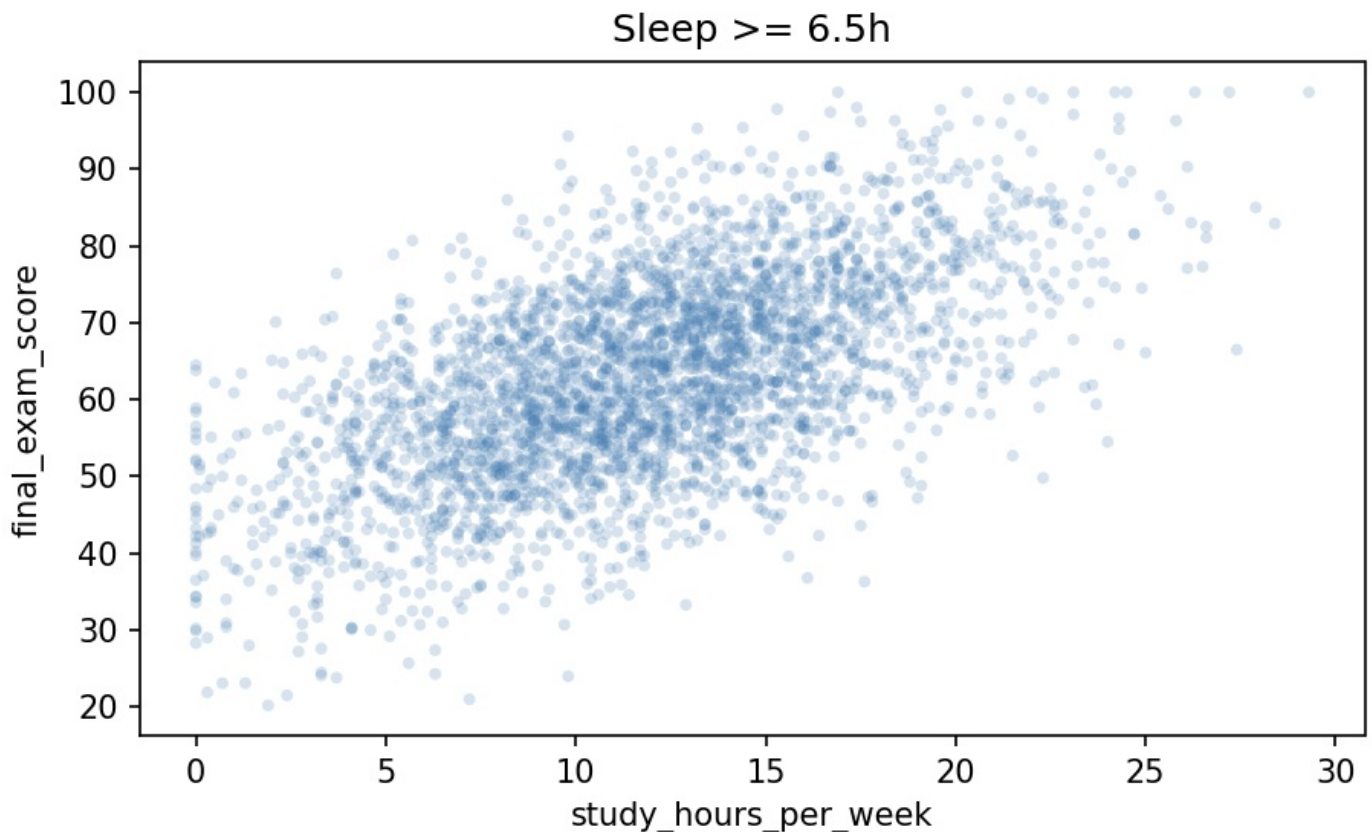


Figure 4: Study hours vs final score for students sleeping 6.5 hours or more.

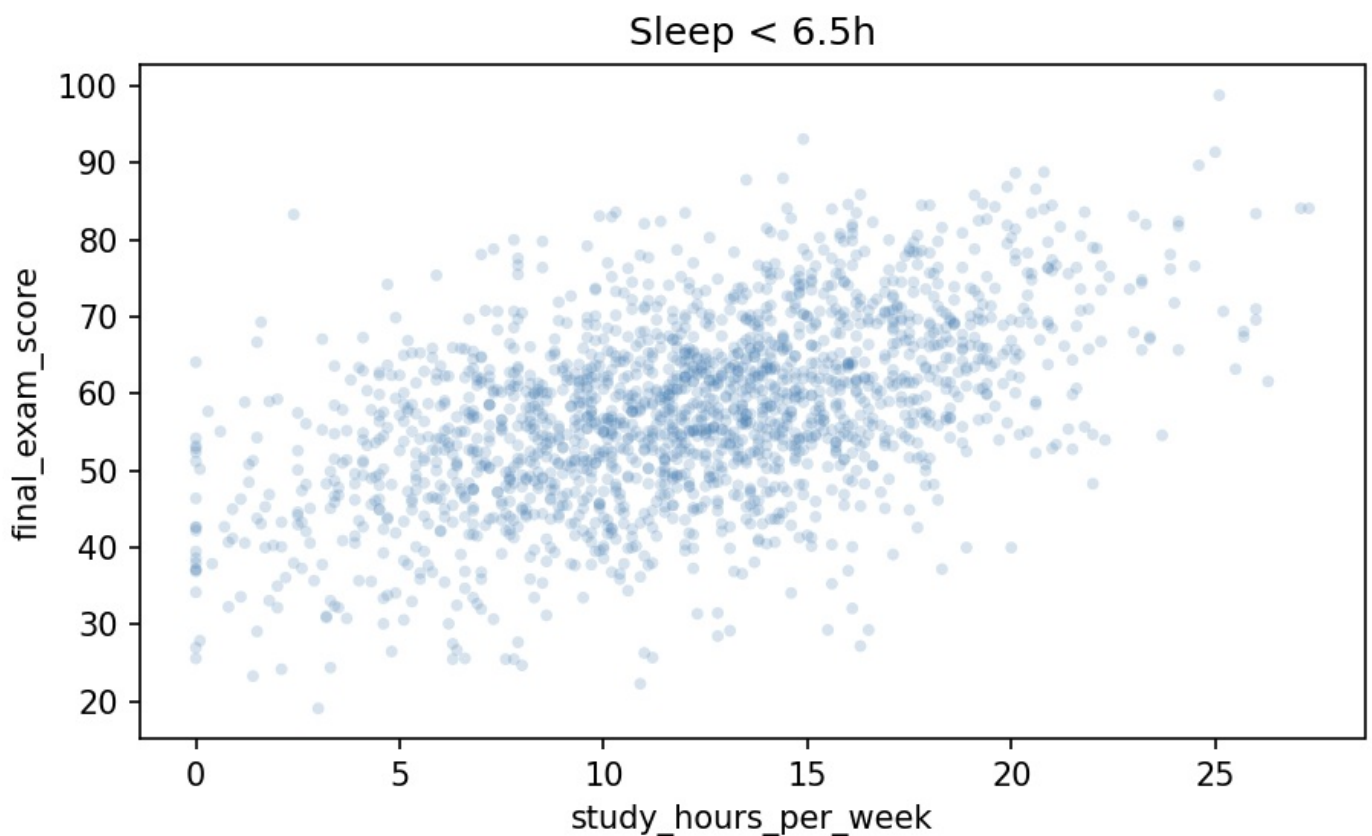


Figure 5: Study hours vs final score for students sleeping less than 6.5 hours.

In Figures 4 and 5 we can draw a linear line on both of them — the more the study hours, the greater the final exam score. But if we take a deeper look, we can see that for students who slept more than 6.5 hours, the steepness of the line is way greater than for those with less than 6.5 hours of sleep. So studying pays more when you sleep well — an interaction between two variables. To test how much this interaction is worth, we gave linear regression one extra hand-made column: study hours multiplied by whether the student sleeps 6.5+ hours. With this column, linear regression improved from 6.84 to 6.64 MAE on the cleaned data (and from 6.63 to 6.40 on the clean data) — the best numbers in our whole study. So human insight plus a simple model beat the automatic pattern-finders.

## Run 2 — clean data

Now we redo the training and testing, but this time with the 100% clean dataset with no errors. The point of this run is to see how much each model changes when the leftover dirt is gone. For multiple linear regression we got an MAE of 6.63 and  $R^2$  of 0.590 — that's a 3.0% change in MAE and 4.4% in  $R^2$ . These aren't big numbers, and the change in MAE wouldn't even be noticeable in real life. SVR gave an MAE of 8.12 and  $R^2$  of 0.39 — a change of 0.7% in MAE and 3.7% in  $R^2$ . Random forest regression gave an MAE of 6.80 and  $R^2$  of 0.563 — a change of 5.9% in MAE and 7.6% in  $R^2$ . The baseline here is 10.32. So the model that changed the least between cleaned and clean data was SVR — but we have to be careful with this observation, because SVR also learned the least (its  $R^2$  is the lowest by far), and a model that learned little has little to lose. The highest accuracy of all models, on both datasets so far, belongs to multiple linear regression. Let's redo this experiment with the raw messy data — the real test.

## Run 3 — raw messy data, and the ranking flips

We trained our models on the raw messy data, no cleaning at all, and here the story changed completely. Linear regression got an MAE of 8.68 and  $R^2$  of 0.320 — its  $R^2$  collapsed from 0.59 to 0.32. SVR got an MAE of 8.89 and  $R^2$  of 0.292. And random forest regression got an MAE of 7.13 and  $R^2$  of 0.564 — almost exactly the same as its clean-data score. The baseline here is 10.68. So the ranking flipped: on clean data linear regression beats the forest (6.63 vs 6.80), but on raw dirty data the forest crushes linear regression (7.13 vs 8.68). The accuracy champion and the robustness champion are two different models. The explanation is in how each model works: linear regression and SVR use the actual feature values, so impossible entries like 150 study hours and our -999 fillers drag their learned weights into wrong places. The forest only asks "above or below a threshold?", so one absurd value barely moves it. The damage from raw data was +2.05 MAE for linear regression, +0.77 for SVR ( $C=100$ ), and only +0.34 for the forest.

## Recovery

Now let's calculate the Recovery for each model, using  $\text{Recovery\%} = (\text{MAE}_{\text{raw}} - \text{MAE}_{\text{messy}}) / (\text{MAE}_{\text{raw}} - \text{MAE}_{\text{clean}}) \times 100$ :

- Multiple linear regression: **90.0%**

- SVR: **92.2%**
- Random forest regression: **not measurable** — its total damage (0.34 MAE) is so small that it is within the normal split-to-split noise, so the ratio becomes unstable (the same formula gave +21% on one split and -17% on another). This is itself a result: the forest took too little damage for recovery to even be meaningful.

So our cleaning repaired around 90% of the damage for the value-based models. The remaining ~10% is information that is permanently destroyed — for example, the medians we filled are not the students' true values, and no cleaning can bring the truth back.

## Limitations

---

This experiment was made on only one dataset, and of course that isn't enough to make a general conclusion. The dataset was AI-generated, and its corruption was injected on purpose — real-world dirt is unknown and can be nastier. The dataset also turned out to have a mostly linear structure, which may have given multiple linear regression a step forward; the ranking could be different on strongly non-linear data. We used a single train/test split with no cross-validation, so small gaps between models (like the 0.26 MAE between forest and linear on cleaned data) are suggestive, not conclusive. Our missing-data handling was a simple median fill, and we showed the missing sleep data isn't random, so this fill carries a small bias. Only 3 models were tested — there may be other models that perform better. And finally, this is one niche only — students — so these results may not apply to all datasets and fields.

## Conclusion

---

Linear regression won accuracy (6.63 clean / 6.84 cleaned / 6.40 with the interaction feature). Random forest won robustness (+0.34 MAE damage on raw data vs linear's +2.05 — the ranking flipped, forest 7.13 beat linear 8.68 raw). And SVR won the recovery category, with 92% of its damage repaired by cleaning (vs linear's 90%; the forest excluded — damage too small to measure). Recovery% ranks how repairable each model's damage was, not model quality, so SVR's win reflects the cleaning pipeline's effectiveness rather than SVR's strength — it remained last in accuracy in every condition.

The takeaway: three categories, three different winners. Model choice depends on data quality and data shape, not sophistication. If you can clean your data well, a simple accurate model like linear regression is the best choice; if you are stuck with dirty data you cannot clean, the robust random forest is the safer one.

## Future Work

---

In the future I want to test on more and different datasets — especially a strongly non-linear one, to see if the accuracy ranking flips — and use cross-validation for more reliable numbers, compare different ways of filling missing data (median vs KNN), and test harder relationships between factors.

## Acknowledgment

---

AI assistance was used for debugging code, verifying results through independent replication, and language editing. All experiments, analysis decisions, and findings are the author's own.